The `PrivacyModel` contains a single handler, `OnGet`, which indicates that it should run in response to `GET` requests for the page. As the method returns `void`, executing the handler executes the associated Razor template for the page to generate the HTML.

NOTE Razor Pages are focused on building page-based apps, so you typically want to return HTML rather than JSON or XML. You can also use an `IActionResult` to return any sort of data, to redirect users to a new page, or to send an error. You'll learn more about `IActionResult`s in chapter 15.

DI is used to inject an `ILogger<PrivacyModel>` instance into the constructor of the page model the same way you would inject a service into a minimal API endpoint handler. The service is unused in this example, but you'll learn all about `ILogger` in chapter 26.

Clearly, the `PrivacyModel` page model doesn't do much in this case, and you may be wondering why it's worth having. If all page models do is tell the Razor Page to generate HTML, why do we need them at all?

The key thing to remember here is that now you have a framework for performing arbitrarily complex functions in response to a request. You could easily update the handler method to load data from the database, send an email, add a product to a basket, or create an invoice—all in response to a simple HTTP request. This extensibility is where a lot of the power in Razor Pages (and the MVC pattern in general) lies.

The other important point is that you've separated the execution of these methods from the generation of the HTML. If the logic changes, and you need to add behavior to a page handler, you don't need to touch the HTML generation code, so you're less likely to introduce bugs. Conversely, if you